Bilan de projet

SAÉ 3.02 — Carrefour intelligent connecté

Groupe « Python Mayo » — Atakan Gocer · Louise Halter-Tisler

1. Bilan des fonctionnalités mises en œuvre

Cette revue confronte, ligne à ligne, le livrable au périmètre du cahier des charges déposé le 24 septembre 2026. Celui-ci classait 24 fonctionnalités selon la méthode MoSCoW. Seules les 16 classées *Must* et *Should*, soit 100 heures estimées, constituaient un engagement.

Synthèse

Catégorie	Prévues	Fait	Partiel	Non fait
Must have	10	10	0	0
Should have	6	5	1	0
Engagé (Must + Should)	**16**	**15**	**1**	**0**
Could have	4	0	1	3
Won't have	4	exclues du périmètre, aucune développée		

Tout le périmètre engagé a été livré, une seule fonctionnalité l'étant partiellement. Sur les quatre *Could have*, une l'est en partie et trois ne l'ont pas été.

Revue fonctionnalité par fonctionnalité

Fonctionnalité	Priorité	Statut	Ce qui a été livré
Moteur de simulation	Must	**Fait**	Simulation à 30 pas par seconde : usagers, trajectoires avec virages, dynamique avec freinage plus fort que l'accélération
Géométrie et Code de la route	Must	**Fait**	Voies entrante et sortante distinctes, arrêt à la ligne, priorité au piéton engagé, refus de priorité en tourne-à-gauche (R415-5), interdiction de bloquer le carrefour
Feux avec phase de dégagement	Must	**Fait**	Cycle vert, orange, « tout rouge » entre deux axes
Protocole applicatif sur sockets TCP	Must	**Fait**	Messages JSON sur une ligne, découpés par un délimiteur, avec conservation du message incomplet (`protocole.py` , 48 lignes)
Serveur multi-clients	Must	**Fait**	Un thread de lecture par client et une file thread-safe ; la même classe sert sur les deux VM
IHM PyQt6	Must	**Fait**	`QGraphicsScene` : routes, usagers orientés, feux, passages piétons, panneaux dynamiques
Thread réseau Qt	Must	**Fait**	`QThread` et `pyqtSignal` ; aucun widget n'est touché hors du thread principal
Persistance MariaDB	Must	**Fait**	Schéma, DAO et historique des usagers. Seuls les accidents sont journalisés dans la table des événements
Centrale V2I	Must	**Fait**	Voie réservée, forçage des feux, déviation des usagers gênants, jamais d'un véhicule déjà engagé

Fonctionnalité	Priorité	Statut	Ce qui a été livré
Alertes anticipées et priorités	Must	**Fait**	Alerte douze secondes avant l'arrivée, niveaux P1 à P3, arbitrage entre urgences concurrentes
Serrage à droite	Should	**Fait**	Décalage latéral, ralentissement, feux de détresse ; trois garde-fous issus de l'observation
Tableau de bord et métriques	Should	**Fait**	Collisions, déviations forcées, temps moyen des urgences et des véhicules ordinaires, alertes en cours
Commandes opérateur	Should	**Fait**	Ajout et retrait de chaque type d'usager, choix du niveau de priorité des alertes
Étude comparative	Should	**Fait**	Cinq configurations mesurées sur des situations de trafic identiques (graine fixée), résultats en CSV et en graphique
Authentification HMAC des alertes	Should	**Fait**	Signature HMAC-SHA256 des alertes du boîtier ambulance ; une alerte à signature invalide est rejetée
Mode dégradé en cas de perte réseau	Should	**Partiel**	Le client détecte la perte, passe en rouge, désactive ses boutons et se reconnecte seul ; la passerelle prévient aussi les clients quand le moteur tombe. La bascule des feux en cycle fixe n'est pas simulée
Export des statistiques	Could	**Partiel**	Fait pour l'étude comparative (CSV et histogramme PNG). Les statistiques de session restent consultables en base, sans export
Comportement des véhicules autonomes	Could	**Non fait**	Ils existent comme type (35 % du trafic, affichés en orange) mais conduisent exactement comme les véhicules ordinaires
Taux de non-respect paramétrable	Could	**Non fait**	Tous les usagers respectent les règles
Latence et pertes réseau simulées	Could	**Non fait**	Seule la coupure franche est gérée, avec reconnexion automatique
Enchaînement de plusieurs carrefours	Won't	Exclu	Respecté : un seul carrefour
Déploiement sur matériel réel	Won't	Exclu	Respecté
Cartographie réelle de Mulhouse	Won't	Exclu	Respecté : carrefour générique
Rendu 3D	Won't	Exclu	Respecté : vue de dessus 2D

Ce qui a changé par rapport au cahier des charges

La répartition sur les machines virtuelles. Le cahier des charges prévoyait une *VM Serveur* portant le moteur, la centrale V2I, le serveur de sockets et la base, et une *VM Client* portant l'interface. Le client ayant demandé de répartir les **processus et les bases**, nous avons scindé le serveur : la VM1 porte le moteur et la base des usagers, la VM2 porte la passerelle réseau et la base des sessions, liées par un socket TCP interne. L'interface tourne désormais dans la VM2. La base unique à trois tables est devenue deux bases.

Le système d'exploitation. Ubuntu 24.04 était prévu ; le déploiement s'est fait sur Debian 13, imposée pour les machines virtuelles.

La gestion de version. Git et GitHub étaient prévus dès le jalon J2, avec une convention de branches. Le dépôt n'a été créé que le 8 novembre, et il n'existe qu'une branche. Avant cette date, les versions circulaient sous forme d'archives. Les messages de commit sont en revanche explicites, comme demandé.

Les tests. Le cahier des charges promettait des scénarios de test rejouables à graine fixée pour les chemins critiques du moteur. Seule l'étude comparative fixe ses graines et rejoue les mêmes situations ; nous n'avons écrit aucun test automatisé avec vérification de résultat.

L'exécution sur un seul poste, prévue comme repli en cas de panne de VM, reste possible en pointant les variables d'environnement vers `127.0.0.1`, mais nous ne la documentons plus : nous avons préféré un unique parcours de déploiement, celui des postes de l'IUT.

Ce qui n'a pas été développé, et pourquoi

Les trois *Could have* restants étaient conditionnés à la livraison complète des *Must* et des *Should*. Une fois celle-ci acquise, le temps disponible a été absorbé par le déploiement réparti, bien plus coûteux que prévu.

Les tests automatisés. C'est la lacune que nous regrettons le plus. Elle nous a interdit de refactorer le code en fin de projet, faute de pouvoir vérifier rapidement que rien n'était cassé.

Le coût réel pour le reste du trafic. Le cahier des charges posait la question « quel gain, et à quel prix pour le reste du trafic ? ». Nous mesurons précisément le gain, mais le prix n'est mesuré qu'en nombre de collisions et de déviations forcées. Nous n'avons pas comparé le temps de traversée des usagers ordinaires d'une configuration à l'autre. C'est la principale limite de nos conclusions.

Des limites que nous avions identifiées nous-mêmes, sans qu'elles figurent au cahier des charges :

- les usagers déviés sont envoyés vers une sortie tirée au hasard, et non vers la branche la moins chargée ;
- la durée d'un couloir est plafonnée à trente secondes, ce qui limite la famine de l'axe perpendiculaire sans la traiter formellement ;
- les distances sont en pixels et le temps en ticks : nos résultats sont des comparaisons relatives, non des durées transposables telles quelles ;
- une réservation d'intersection remplacerait les garde-fous empiriques actuels et supprimerait probablement les collisions résiduelles, environ cinq pour 12 000 ticks.

---

2. Conclusion technique

Les difficultés rencontrées, et comment nous les avons levées

La préemption des feux ne suffisait pas — et faisait pire. Notre première solution forçait le feu au vert à l'approche d'une ambulance. Les mesures ont montré qu'elle était **moins bonne que ne rien faire** (46,3 s contre 31,8 s). La raison : le feu vert retient le trafic perpendiculaire mais ne vide pas la file déjà présente sur la voie de l'ambulance, qui se retrouve bloquée derrière des véhicules ayant le vert et nulle part où aller. C'est ce constat qui a fait naître la voie réservée, avec déviation active des usagers gênants.

Nos chiffres ne prouvaient pas ce que nous affirmions. Nous attribuions le gain à la voie réservée sans l'avoir isolé de celui de la préemption des feux. Nous avons construit une étude par ablation : cinq configurations, chacune ajoutant un mécanisme, mesurées sur des situations de trafic identiques. C'est elle qui a révélé le résultat contre-intuitif ci-dessus.

Les collisions. Les premières versions en produisaient plus de 2 400 pour 12 000 ticks. Le travail sur les règles de priorité — trajectoires sécantes, refus de priorité en tourne-à-gauche, interdiction de bloquer le carrefour — les a ramenées à environ cinq. Les cas restants surviennent dans des configurations de croisement serrées ; les supprimer demanderait une réservation d'intersection.

Les interblocages du serrage à droite. Quand un usager se range pour une ambulance elle-même bloquée plus loin, tout le monde s'attend mutuellement. Nous avons ajouté un garde-fou : au-delà de cinq secondes, l'usager cesse de céder le passage à cette ambulance. La valeur est empirique, issue de l'observation, et nous l'assumons comme telle.

Le découpage en deux processus. Séparer le moteur de la passerelle a soulevé un problème que le montage monolithique masquait : quand la passerelle transmet une commande au moteur, comment la réponse revient-elle au **bon** client ? Nous avons étiqueté chaque commande avec l'identifiant de son émetteur, que le moteur renvoie dans sa réponse. Cette étiquette est purement interne : elle ne quitte jamais la passerelle en direction des clients.

Le réseau entre machines virtuelles. C'est ce qui nous a coûté le plus de temps au déploiement. Les VM fournies n'avaient qu'un seul profil réseau, rattaché à aucune carte : chaque tentative de réparer une interface faisait perdre son adresse à l'autre. Nous avons fini par créer un profil par carte, avec des adresses fixes, ce qui a supprimé le problème à la racine.

Des bugs invisibles à la lecture. Trois défauts n'ont été découverts qu'en déployant réellement : un attribut inexistant dans un chemin de code jamais exécuté sans base de données, une détection de carte réseau qui configurait la mauvaise tout en paraissant correcte, et un message affirmant un enregistrement qui n'avait pas eu lieu. Ils sont détaillés dans l'IA-graphie.

Ce que nous ferions autrement

Écrire des tests dès le début. C'est notre principal regret. Sans eux, chaque validation demande un déploiement complet et une manipulation à la main. En fin de projet, cela nous a interdit de toucher au code pour l'améliorer, le risque étant disproportionné.

Transmettre les contraintes avant de produire du code. Le prototype Pygame a été entièrement réécrit parce que nous n'avions pas communiqué le cahier des charges assez tôt.

Déployer plus tôt en conditions réelles. Les bugs les plus coûteux ne se manifestaient que sur les VM, avec une base de données réellement joignable. Un déploiement dès les premières semaines les aurait révélés quand il restait du temps.

Ce que nous referions à l'identique

Le choix de la voie réservée plutôt que la seule préemption des feux. C'est ce qui fait l'intérêt du projet, et la mesure le confirme.

L'étude comparative par ablation. Elle transforme une affirmation en démonstration, et c'est elle qui donne de la valeur à nos chiffres.

La séparation stricte des rôles. Un seul composant décide, les autres transmettent et affichent. Cette discipline a rendu le découpage sur deux machines presque mécanique le jour où il a fallu le faire.

---

3. Conclusion organisationnelle

Notre méthode de travail

Nous avons réalisé **l'ensemble du projet ensemble**, sans répartir les modules par personne : le code, les tests de déploiement et la documentation. Ce choix nous a permis de comprendre tous les deux l'intégralité du projet, ce qui est nécessaire pour une soutenance où chacun peut être interrogé sur n'importe quelle partie. Son revers est qu'il suppose de se retrouver aux mêmes moments, ce que nos emplois du temps différents ont compliqué (voir ci-dessous).

Les rôles de cabinet conseil et de cabinet développement, évoqués dans la fiche de cadrage, n'ont pas été distingués en pratique.

Des emplois du temps différents

Nous ne sommes pas dans le même groupe de travaux pratiques : Atakan relève de RoM et Louise de DevCloud. Nos séances d'autonomie n'étaient donc pas les mêmes. Atakan, en alternance du 5 octobre au 2 novembre, n'en avait aucune sur cette période, alors que Louise en disposait. Cette asymétrie a pesé sur le rythme du projet, et une part importante du travail d'Atakan s'est faite en dehors des séances.

L'usage de l'IA dans notre organisation

Nous avons utilisé Claude tout au long du projet. Cela a changé notre façon de travailler : moins de temps passé à écrire du code, davantage à spécifier ce que nous voulions, à tester, et à comprendre les réponses obtenues. L'IA-graphie détaille cet usage, y compris ce qu'il nous a coûté.

Un effet que nous n'avions pas anticipé : notre contribution la plus décisive a été **le test en conditions réelles**. Trois bugs qu'aucune relecture n'aurait détectés ne sont apparus qu'en déployant sur les machines virtuelles.

---

4. Analyse du planning

Le planning prévisionnel

Le cahier des charges déposé le **24 septembre** prévoyait sept jalons répartis des semaines 39 à 46, pour une charge estimée à **100 heures de groupe** — 72 heures pour le périmètre *Must*, 28 pour le *Should*.

Le temps réellement consommé

	Volume de groupe
Estimation du cahier des charges	100 h
Séances d'autonomie encadrées	**44 h**
Travail personnel hors séances	**≈ 60 h** (estimation)
Total réalisé	**≈ 104 h**

Les séances d'autonomie inscrites à l'emploi du temps se sont étalées de la semaine 37 à la semaine 46, à raison de trois à dix heures par semaine selon les périodes. Le chiffre du travail personnel est une estimation : nos comptes rendus hebdomadaires relevaient les tâches accomplies plus systématiquement que les heures passées.

L'analyse de l'écart

L'estimation globale était juste. 104 heures réalisées pour 100 prévues : à quatre heures près, nous avions correctement évalué l'ampleur du travail. C'est le point positif de notre planning, et il mérite d'être souligné.

C'est la répartition qui était fausse. Près de **60 % du projet a été réalisé en dehors des séances encadrées**. Avec seulement 44 heures inscrites à l'emploi du temps face à une charge estimée à 100, le recours massif au travail personnel n'était pas un dépassement accidentel : il était **arithmétiquement inévitable dès le départ**. Notre planning ne l'énonçait nulle part, et c'est sa véritable faiblesse — il annonçait un volume sans préciser où ces heures seraient prises.

L'estimation ne chiffrait que la production. Nous avions évalué le temps d'écrire le code, pas celui de comprendre le besoin, de revenir sur une décision, ni de mettre en production. Ce dernier poste s'est révélé le plus coûteux : deux journées entières de déploiement, pour des raisons sans rapport avec notre code — profils réseau des machines virtuelles, droits d'administration, variables d'environnement graphiques.

Le déséquilibre est resté invisible. Comme la charge était absorbée en dehors des séances, elle n'apparaissait pas comme un retard : les jalons étaient tenus. Un suivi qui n'aurait regardé que les dates de livraison aurait conclu que tout allait bien, sans voir que plus de la moitié du travail se faisait hors du cadre prévu.

Ce que nous en retirons. Estimer un volume ne suffit pas, il faut préciser **où** les heures seront prises. Annoncer 100 heures quand l'emploi du temps n'en offre que 44 revient à sous-entendre un travail personnel que nous n'avions ni chiffré ni annoncé. Un planning honnête aurait dit : « 44 heures encadrées, environ 56 heures à prendre sur notre temps personnel » — ce qui aurait permis d'en discuter dès le départ, et de réduire le périmètre si ce volume n'était pas tenable.

Une estimation doit aussi inclure ce qui n'est pas de la production : lire un cahier des charges, revenir sur une décision, installer, déboguer, documenter. Ces activités ont pesé plus lourd que l'écriture du code elle-même.

Le déroulement réel face aux jalons

Jalon	Prévu	Réalisé	Écart
J1 — Cahier des charges	24 sept.	Déposé le 24 sept.	Tenu
J2 — Environnement (VM, réseau, MariaDB, dépôt Git)	S40	VM, réseau et base en septembre ; **dépôt Git le 8 novembre**	Cinq semaines de retard sur le dépôt
J3 — Noyau de simulation	S42	Dès la semaine 38	Environ quatre semaines d'avance
J4 — Sockets et IHM PyQt6	S44	Vers le 18 sept., semaine 38	Environ six semaines d'avance
J5 — Priorisation V2I complète	S45	Semaines 39 et 40, d'après l'historique de conception	Environ cinq semaines d'avance
J6 — Tests, mesures, corrections	S46	Mesures comparatives en septembre ; corrections de déploiement en novembre ; **aucun test automatisé**	Mesures en avance, tests non réalisés
J7 — Livraison finale	15 nov.	Dépôt, documentation, bilan, IA-graphie	Soutenance à confirmer

Le Gantt prévoyait un effort réparti de la semaine 39 à la semaine 46, avec le pic de priorisation V2I en semaine 43, celui des tests en semaine 45 et celui de la documentation en semaine 46. La charge s'est en réalité concentrée en **deux blocs** : les semaines 37 à 40, où l'essentiel du développement a eu lieu, puis les semaines 45 et 46, consacrées au déploiement et à la documentation.

Entre les deux, la logique métier n'a pas évolué : le code livré correspond à l'état du 2 octobre. Atakan était en alternance du 5 octobre au 2 novembre, sans séance d'autonomie, alors que Louise en disposait sur cette période.

Les écarts et leurs causes

La réécriture de la mi-septembre. Le prototype Pygame a été entièrement refait en PyQt6 avec des sockets. La cause est identifiable : nous avons commencé à produire du code avant d'avoir lu et transmis le cahier des

charges. La logique métier a pu être conservée, ce qui a limité la perte, mais toute la couche technique était à refaire.

Une avance qui a masqué un retard. Le périmètre fonctionnel était complet dès le 2 octobre, cinq semaines avant le jalon. Cette avance nous a donné le sentiment que le reste suivrait, alors que le plus lourd restait à faire : le déploiement réparti, exigence structurante du sujet.

La répartition sur deux VM, tardive. Elle n'est intervenue qu'au 8 novembre. Jusque-là, le projet tournait sur une seule machine. Ce report a concentré en fin de projet un travail d'architecture et de déploiement qui a révélé des bugs, ceux-là mêmes qui nous ont pris le plus de temps. Les découvrir en septembre aurait été nettement plus confortable.

Le déploiement, largement sous-estimé. Nous l'avions prévu comme une formalité. Il a occupé plusieurs journées, pour des raisons étrangères à notre code : profils réseau des machines virtuelles, droits `sudo`, variables d'environnement graphiques. C'est l'enseignement principal de notre planning : **le temps de mise en production ne se devine pas, il s'éprouve**.

Un dépôt tardif. Faute de dépôt Git avant le 8 novembre, l'historique du dépôt ne reflète que les dernières semaines du projet, non sa durée réelle. C'est aussi ce qui nous a privés du suivi de version que le cahier des charges annonçait pour le jalon J2.

La documentation, concentrée sur les derniers jours. Rédigée au fil de l'eau, elle aurait été plus fidèle, notamment sur les décisions de conception dont nous avons dû retrouver les raisons a posteriori.

La gestion des risques, avec le recul

Risque prévu	S'est-il produit ?	Mesures prévues	Ce qui s'est passé
Retard sur le moteur de simulation	Non	Jalon J3, périmètre *Could* sacrifiable	Moteur terminé dès la semaine 38
Conflits et collisions entre usagers	**Oui**, au début : plus de 2 400 collisions pour 12 000 pas	Longues campagnes de tests, scénarios rejouables	Ramenées à environ cinq ; la graine fixée de l'étude comparative a permis de rejouer les mêmes situations, mais sans tests automatisés
Panne ou perte de temps sur les VM	**Oui**, plusieurs journées perdues au déploiement	Dépôt distant, instantanés de VM, exécution sur un seul poste	Dépôt créé tardivement, instantanés non utilisés, mode mono-poste possible mais non documenté

Le risque le plus sérieusement sous-estimé est donc le troisième : nous l'avions identifié, mais les mesures censées le réduire n'ont été appliquées que partiellement.